

HDOC - Day 9 Activity

C Programming Assignment

Aim

To prepare flowcharts, algorithms, pseudocode, test cases, C programs, compilation commands, execution results, and testing for the three menu-driven problems given in HDOC.

Program 1:

Write a menu-driven C program to input two numbers and swap them (i)using a third variable & (ii)without using third variable. Also display the values before and after swapping.

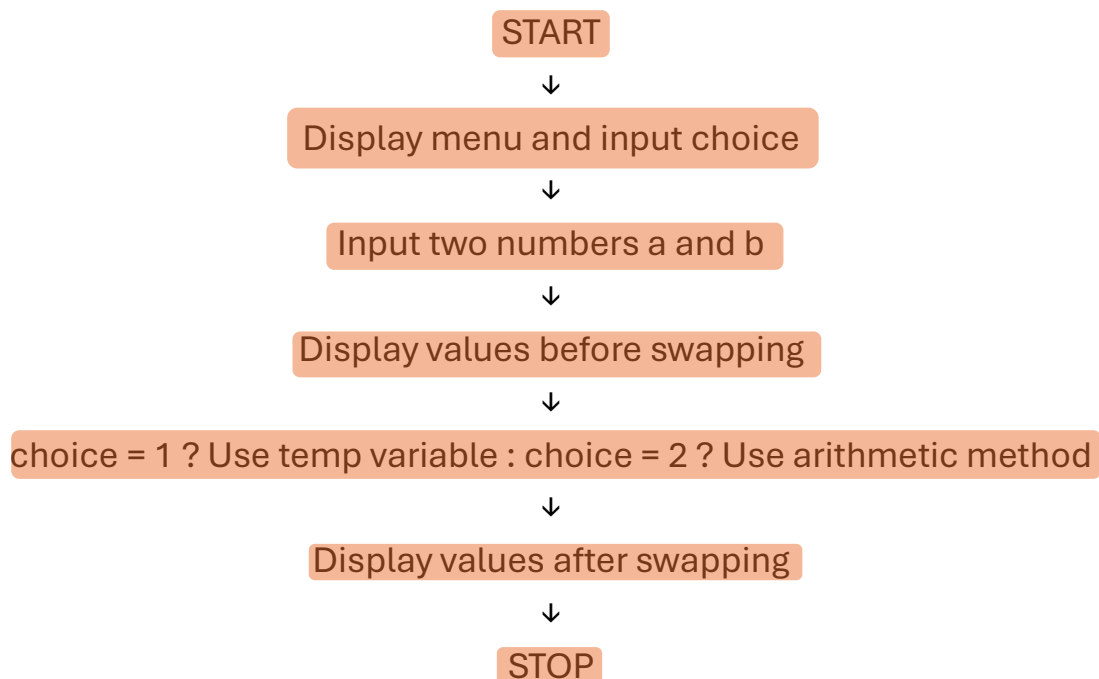
Algorithm

1. Start.
2. Display menu: (i) for third-variable method, (ii) for without-third-variable method.
3. Read choice and two numbers a and b.
4. Display a and b before swapping.
5. If choice =(i), use $\text{temp} = a$; $a = b$; $b = \text{temp}$.
6. If choice =(ii), use $a = a + b$; $b = a - b$; $a = a - b$.
7. Otherwise display Invalid choice and stop.
8. Display a and b after swapping.
9. Stop

Pseudo Code

```
START
DISPLAY menu
INPUT choice
INPUT a, b
DISPLAY a, b before swapping
IF choice = 1 THEN
temp ← a
a ← b
b ← temp
ELSE IF choice = 2 THEN
a ← a + b
b ← a - b
a ← a - b
ELSE
DISPLAY "Invalid choice"
STOP
END IF
DISPLAY a, b after swapping
STOP
```

Flowchart



Standard Test Case Table

Case	Input	Choice	Expected output	Result
1	a=10 , b=25	1	a=25 , b=10	Pass
2	a=-4 , b=7	2	a=7 , b=-4	Pass
3	a=5.5 , b=2.5	1	a=2.5 , b=5.5	Pass
4	a=10 , b=20	3	Invalid choice	Pass

C Code

```
#include <stdio.h>
int main() {
int choice;
double a, b, temp;
printf("1. Swap using third variable\n");
printf("2. Swap without third variable\n");
printf("Enter choice: ");
scanf("%d", &choice);
printf("Enter two numbers: ");
scanf("%lf %lf", &a, &b);
printf("Before swapping: a = %.2f, b = %.2f\n", a, b);
switch(choice) {
case 1:
temp = a; a = b; b = temp;
break;
case 2:
a = a + b; b = a - b; a = a - b;
break;
default:
printf("Invalid choice.\n");
return 0;
}
printf("After swapping: a = %.2f, b = %.2f\n", a, b);
return 0;
}
```

Compilation, Execution and Testing

```
Session Actions Edit View Help
(kali㉿kali)-[~]
└─$ vi swap.c
(kali㉿kali)-[~]
└─$ cc swap.c
(kali㉿kali)-[~]
└─$ ./a.out
1. Swap using third variable
2. Swap without third variable
Enter choice: 1
Enter two numbers: 10 25
Before swapping: a = 10.00, b = 25.00
After swapping: a = 25.00, b = 10.00

(kali㉿kali)-[~]
└─$ ./a.out
1. Swap using third variable
2. Swap without third variable
Enter choice: 2
Enter two numbers: -4 7
Before swapping: a = -4.00, b = 7.00
After swapping: a = 7.00, b = -4.00

(kali㉿kali)-[~]
└─$ ./a.out
1. Swap using third variable
2. Swap without third variable
Enter choice: 1
Enter two numbers: 5.5 2.5
Before swapping: a = 5.50, b = 2.50
After swapping: a = 2.50, b = 5.50

(kali㉿kali)-[~]
└─$ ./a.out
1. Swap using third variable
2. Swap without third variable
Enter choice: 3
Enter two numbers: 10 20
Before swapping: a = 10.00, b = 20.00
Invalid choice.
```

Result: The program compiled successfully and both swapping methods produced the expected results for the prepared test cases.

Program 2:

Write a menu-driven C program to find the (i)sum of the first n natural numbers and (ii)sum of squares of the first n natural numbers.

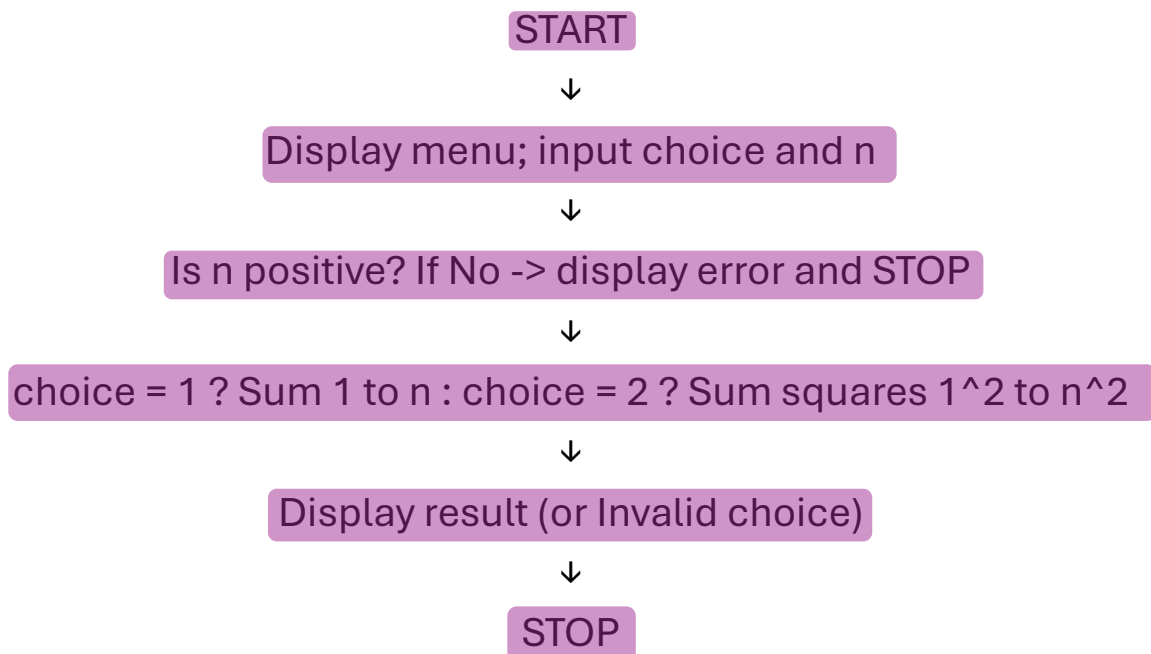
Algorithm

1. Start.
2. Display menu: 1 for sum of natural numbers, 2 for sum of squares.
3. Input choice and n.
4. If $n \leq 0$, display an error and stop.
5. If choice = 1, add integers from 1 to n.
6. If choice = 2, add $i \times i$ for i from 1 to n.
7. Display the calculated result.
8. If choice is invalid, display Invalid choice.
9. Stop.

Pseudocode

```
START
DISPLAY menu
INPUT choice, n
IF n <= 0 THEN DISPLAY error; STOP
IF choice = 1 THEN
  sum ← 0
  FOR i ← 1 TO n: sum ← sum + i
  DISPLAY sum
ELSE IF choice = 2 THEN
  sumSq ← 0
  FOR i ← 1 TO n: sumSq ← sumSq + i*i
  DISPLAY sumSq
ELSE DISPLAY "Invalid choice"
STOP
```

Flowchart



Standard Test Case Table

Case	Input(n)	Choice	Expected output	Result
1	10	1	Sum= 55	Pass
2	5	2	Sum of squares= 55	Pass
3	1	1	Sum= 1	Pass
4	3	3	Invalid choice	Pass

C Code

```
#include <stdio.h>
int main() {
    int choice, n, i;
    long long sum = 0, sumSq = 0;
    printf("1. Sum of first n natural numbers\n");
    printf("2. Sum of squares of first n natural numbers\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    printf("Enter n: ");
    scanf("%d", &n);
    if (n <= 0) { printf("n must be a positive integer.\n"); return 0; }
    switch(choice) {
        case 1:
            for (i = 1; i <= n; i++) sum += i;
            printf("Sum = %lld\n", sum);
            break;
        case 2:
            for (i = 1; i <= n; i++) sumSq += (long long)i * i;
            printf("Sum of squares = %lld\n", sumSq);
            break;
        default:
            printf("Invalid choice.\n");
    }
    return 0;
}
```

Compilation, Execution and Testing

```
Session Actions Edit View Help
(kali㉿kali)-[~]
$ vi natural.c
(kali㉿kali)-[~]
$ cc natural.c
(kali㉿kali)-[~]
$ ./a.out
1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter choice: 1
Enter n: 10
Sum = 55
(kali㉿kali)-[~]
$ ./a.out
1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter choice: 2
Enter n: 5
Sum of squares = 55
(kali㉿kali)-[~]
$ ./a.out
1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter choice: 1
Enter n: 1
Sum = 1
(kali㉿kali)-[~]
$ ./a.out
1. Sum of first n natural numbers
2. Sum of squares of first n natural numbers
Enter choice: 3
Enter n: 3
Invalid choice.
```

Program 3:

Write a menu-driven C program to calculate simple interest, compound amount, compound interest, and total payable amount for given principal, rate, and time.

Formulae Used

Simple Interest: $SI = (P \times R \times T) / 100$

Total Payable under Simple Interest: $Total = P + SI$

Compound Amount: $A = P(1 + R/100)^T$

Compound Interest: $CI = A - P$

Assumption: compounding is annual and time T is in years.

Algorithm

1. Start and display four menu options.
2. Input choice, principal P, rate R and time T.
3. Reject negative input values.
4. Calculate SI, compound amount A, CI, and simple-interest total P + SI.
5. Use switch(choice) to display the requested value.
6. For any other choice, display Invalid choice.
7. Stop.

Pseudocode

START

DISPLAY 1.SI 2.Compound Amount 3.CI 4.Total Payable

INPUT choice

INPUT P, R, T

IF any value < 0 THEN DISPLAY error; STOP

$SI \leftarrow P * R * T / 100$

$A \leftarrow P * (1 + R / 100)^T$

$CI \leftarrow A - P$

$Total \leftarrow P + SI$

SWITCH choice

1: DISPLAY SI

2: DISPLAY A

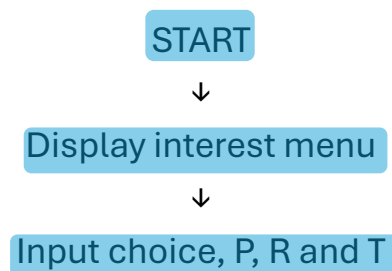
3: DISPLAY CI

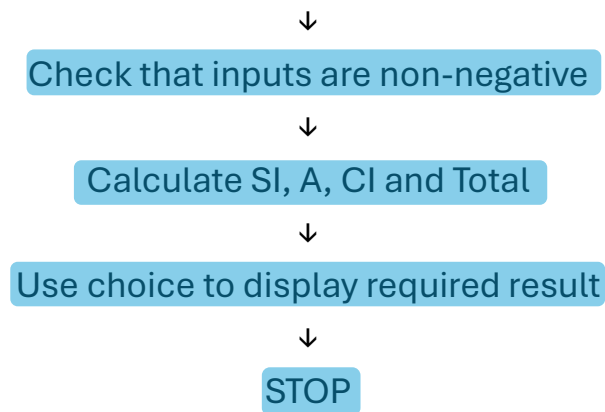
4: DISPLAY Total

default: DISPLAY "Invalid choice"

STOP

Flowchart





Standard Test Case Table

Case	Input(P,R,T)	Choice	Expected output	Result
1	10000,5,2	1	Simple Interest=1000	Pass
2	10000,10,2	2	Compound Amount=12100	Pass
3	10000,10,2	3	Compound Interest=2100	Pass
4	10000,5,2	4	Total Payable Amount=11000	Pass

C Code

```

#include <stdio.h>
#include <math.h>
int main() {
    int choice;
    double p, r, t, si, amount, ci, total;
    printf("1. Simple Interest\n");
    printf("2. Compound Amount\n");
    printf("3. Compound Interest\n");
    printf("4. Total Payable Amount (Simple Interest)\n");
    printf("Enter choice: ");
    scanf("%d", &choice);
    printf("Enter principal, rate (%%), and time (years): ");
    scanf("%lf %lf %lf", &p, &r, &t);
    if (p < 0 || r < 0 || t < 0) { printf("Values cannot be negative.\n"); return 0; }
    si = (p * r * t) / 100.0;
    amount = p * pow(1.0 + r / 100.0, t);
    ci = amount - p;
    total = p + si;
  }

```

```

switch(choice) {
case 1: printf("Simple Interest = %.2f\n", si); break;
case 2: printf("Compound Amount = %.2f\n", amount); break;
case 3: printf("Compound Interest = %.2f\n", ci); break;
case 4: printf("Total Payable Amount = %.2f\n", total); break;
default: printf("Invalid choice.\n");
}
return 0;
}

```

Compilation, Execution and Testing

```

Session Actions Edit View Help
$ cc interest.c -lm

(kali㉿kali)-[~]
$ ./a.out
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount (Simple Interest)
Enter choice: 1
Enter principal, rate (%), and time (years): 10000 5 2
Simple Interest = 1000.00

(kali㉿kali)-[~]
$ ./a.out
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount (Simple Interest)
Enter choice: 2
Enter principal, rate (%), and time (years): 10000 10 2
Compound Amount = 12100.00

(kali㉿kali)-[~]
$ ./a.out
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount (Simple Interest)
Enter choice: 3
Enter principal, rate (%), and time (years): 10000 10 2
Compound Interest = 2100.00

(kali㉿kali)-[~]
$ ./a.out
1. Simple Interest
2. Compound Amount
3. Compound Interest
4. Total Payable Amount (Simple Interest)
Enter choice: 4
Enter principal, rate (%), and time (years): 10000 5 2
Total Payable Amount = 11000.00

```